数字逻辑与微处理器系统设计

实验1: Introduction to Development Tools

韩凯宁 副教授

邮箱: knhan@uestc.edu.cn

办公室: 主楼B3-509

通信抗干扰技术国家级重点实验室

实验课概述

实验课环节共32学时，分8次课完成，规划如下：

第一次课：掌握FPGA行为级仿真能力，完成加法器测试仿真；

第二次课：掌握FPGA上板调试能力，完成基于状态机的倒计时器上板；

第三次课：实验1&实验2分组验收；

第四、五次课： ALU设计、实现、优化，以及验收；

第六、七、八次课： MCU设计、实现、优化，以及验收；

目录

课程设计介绍

- 课程设计1：ALU的设计与实现
- 课程设计2：MCU及其外围电路的设计与实现
- 提交内容：设计报告、源代码、测试程序及测试结果。抄袭将取消考试资格。
- 考核方式：竞赛式考核＋应用设计（排序、数字信号处理)
- 3人一组组队完成，设组长1人，组员2人。教学团队给出设计的平均成绩，由组长分配每个成员课程设计成绩；如2名团队成员对组长分配有异议，可向教学团队提出复议，根据实际情况重新分配。
- 考核时间结课后一周，报告提交时间另行通知。

课程设计介绍

竞赛式考核

- 基于所设计MCU完成两个应用，分别按照速度和效率完成排名
- 每个榜单第一名96分，第二名92分，以此类推。
- 根据提交课程设计报告质量±4分。
- 每个榜单第一名从其余榜单删除，屠榜队从所有榜单删除。

赛题1		赛题2	
速度榜单	效率榜单	速度榜单	效率榜单
第一名	第一名	第一名	第一名
第二名	第二名	第二名	第二名
……	……	……	……

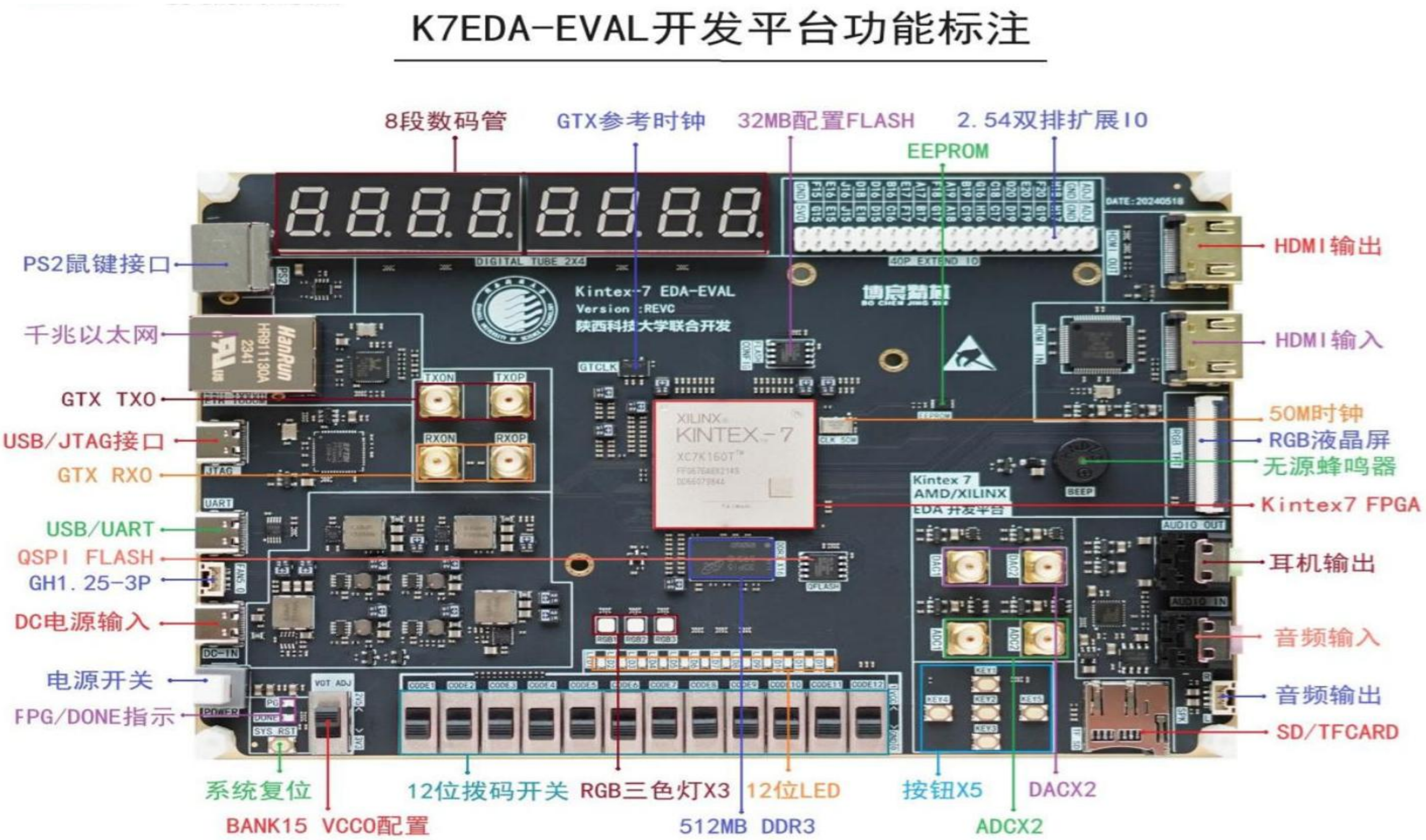
课程设计介绍

考核注意事项

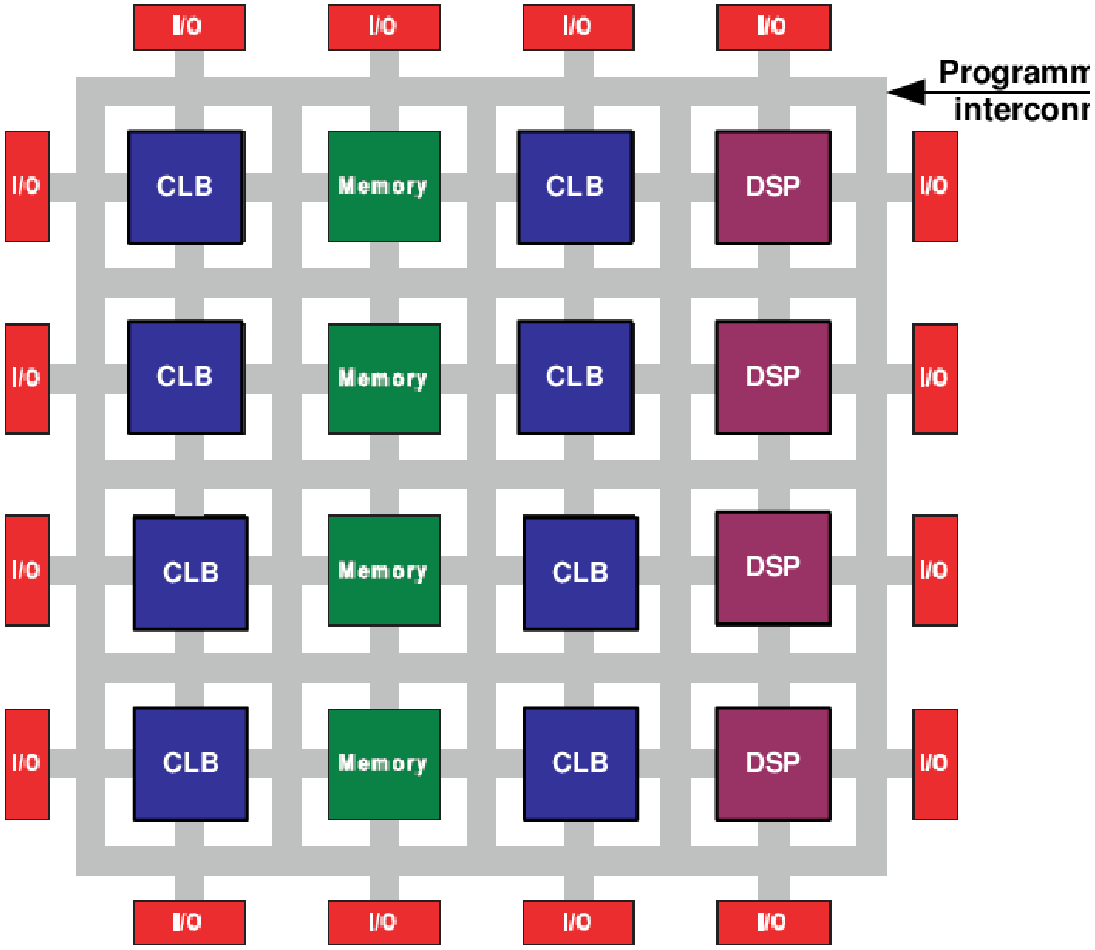
- 处理速度统计方法：完成所有计算结果的时钟频率/时钟周期数，其中时钟频率取时序报告无违例频率与上板时钟频率的较小值；
- 处理效率统计方法：处理速度/硬件开销，其中硬件开销折算方法为6*LUT+10*FF；
- MCU只能执行ARM指令集范围内的指令，不允许额外扩展指令；
- MCU微架构不限，流水、并行、单指令多操作等任意优化均可使用；
- 教学团队会提供板级Testbench，也可以自行搭建测试环境；
- 加分项：采用创新技术优化MCU性能指标，均可作为加分项，分值由教学团队共同评定

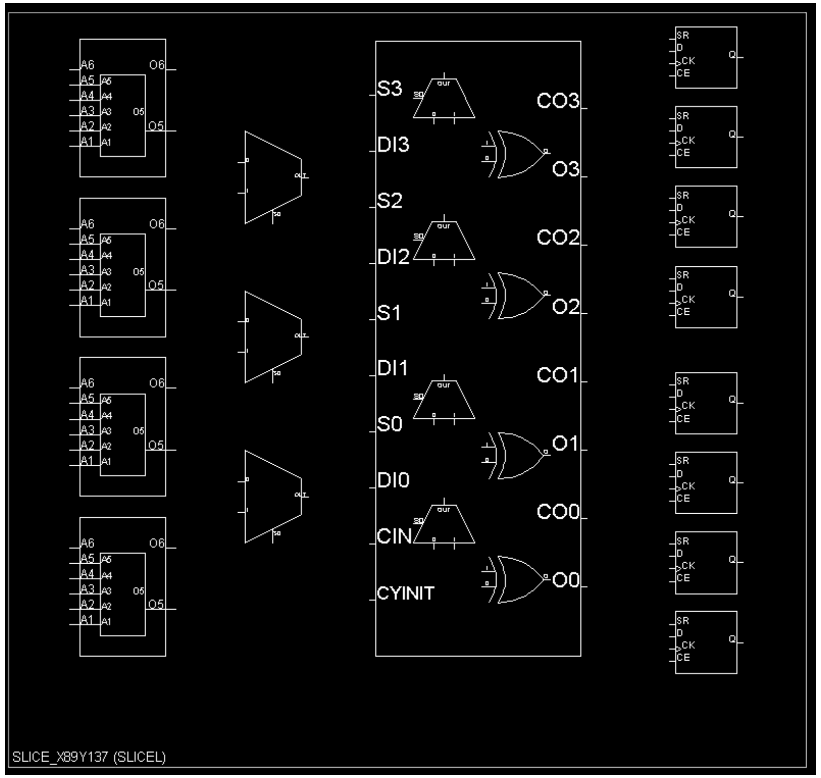
FPGA开发板简介

Xilinx Kintex®-7 FPGA 芯片 XC7K160T-2FFG676-I

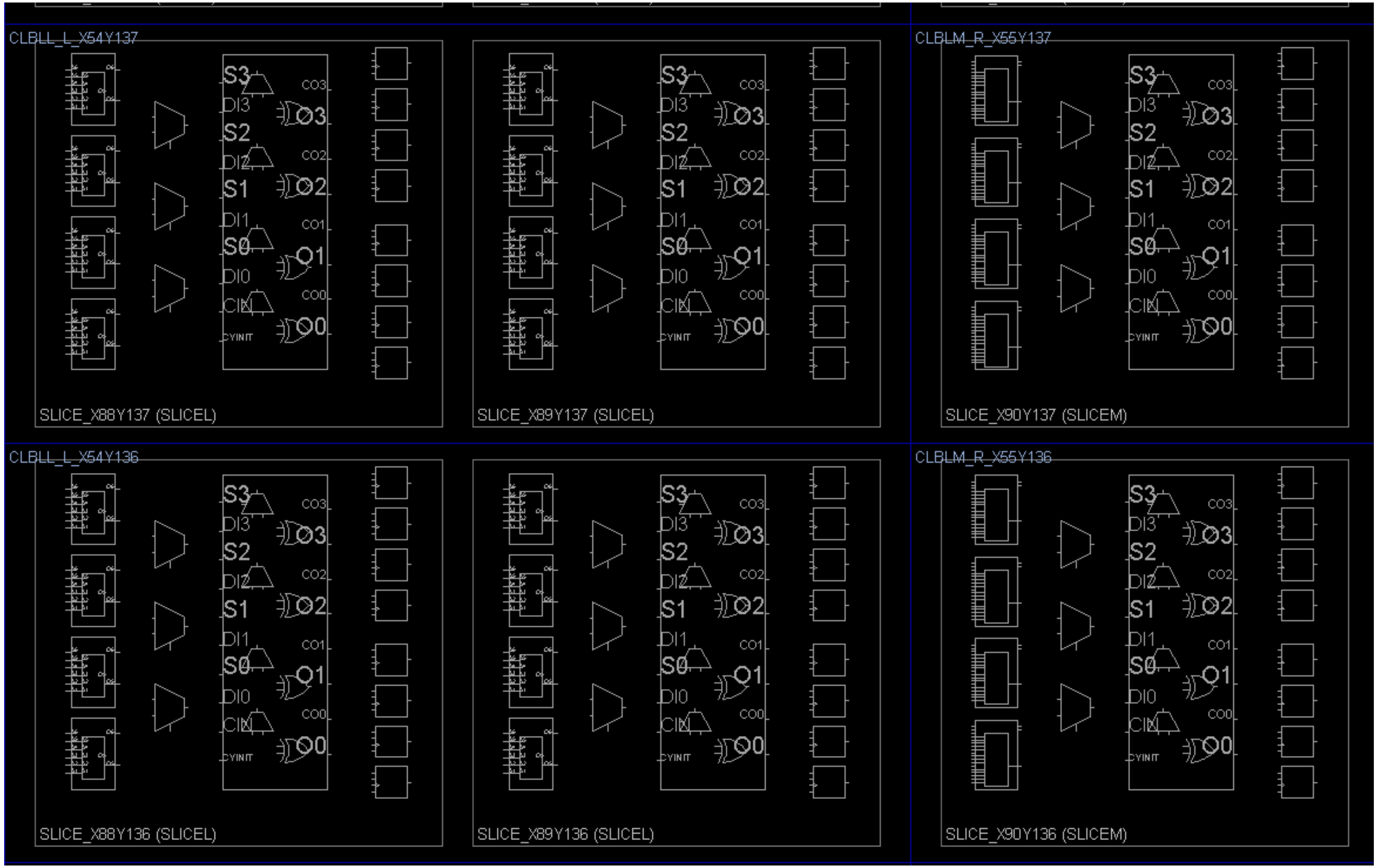


FPGA开发板简介


I/O
Programm
interconn
CLB
Memory
DSP


A6
A5
A4
A3
A2
A1
O6
O5
S3
DI3
S2
DI2
S1
DI1
S0
DI0
CIN
CYINIT
CO3
O3
CO2
O2
CO1
O1
CO0
O0
SR
D
CK
CE
Q
SLICE_X89Y137 (SLICEL)

FPGA开发板简介



CLBLL_L_X54Y137
SLICE_X88Y137 (SLICEL)
SLICE_X89Y137 (SLICEL)
CLBLM_R_X55Y137
SLICE_X90Y137 (SLICEM)
CLBLL_L_X54Y136
SLICE_X88Y136 (SLICEL)
SLICE_X89Y136 (SLICEL)
CLBLM_R_X55Y136
SLICE_X90Y136 (SLICEM)

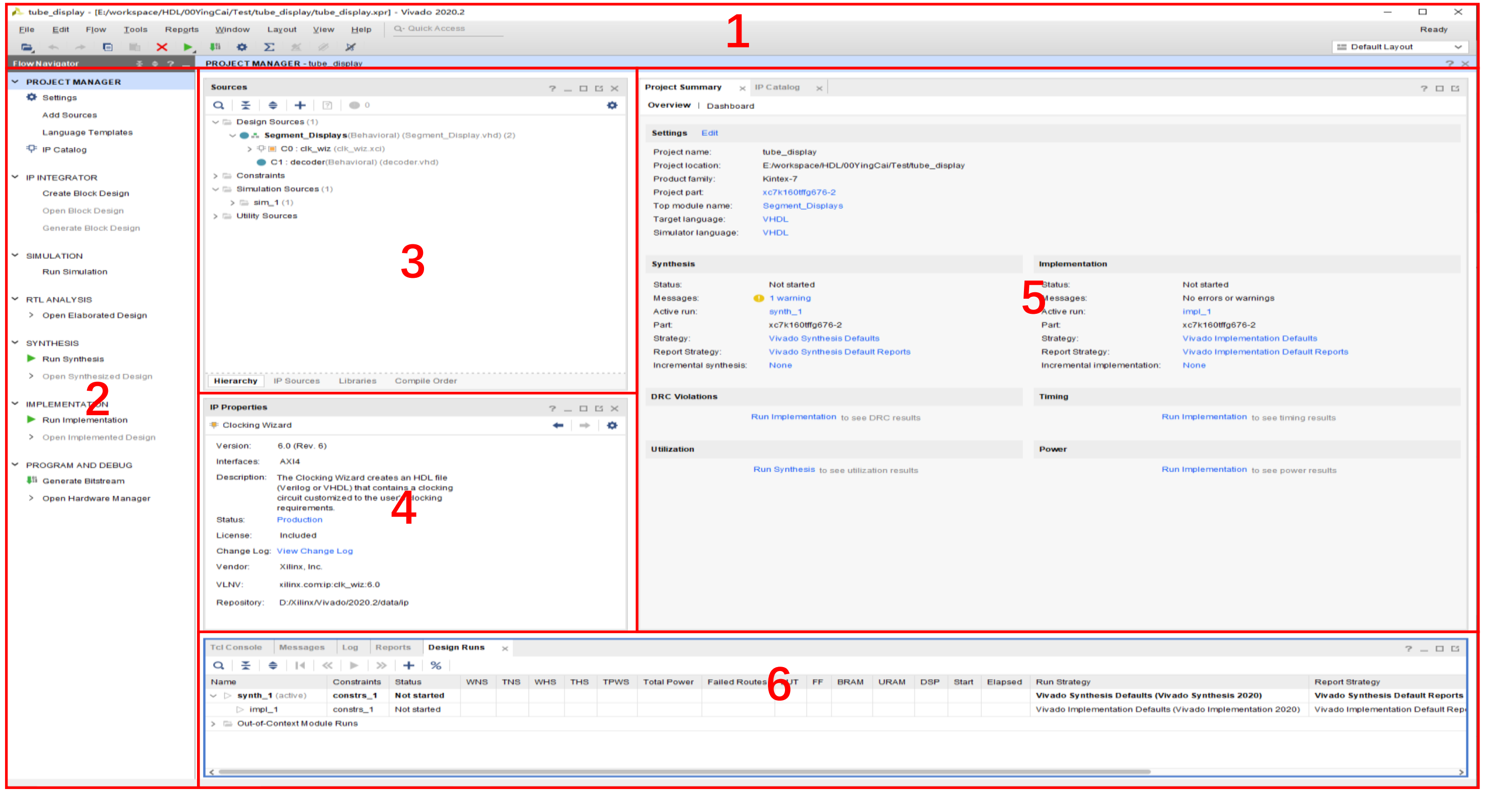
Vivado开发工具简介

1区域中常用的功能有： Quick Start中的Create Project和Open Project以及Tasks中的Open Hardware Manager

2区域中是最近打开过的工程

3区域是使用Tcl命令的操作区域

Vivado开发工具简介

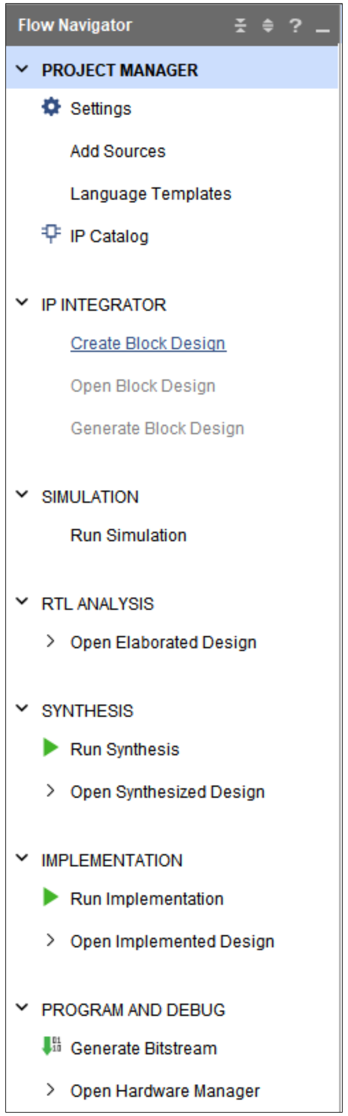


Vivado的主界面分为以下的6大部分，下面做介绍。

Vivado开发工具简介

工具栏：大部分常用功能在其它区域都有按钮，使用起来更加方便。工具栏内容较多，大家感兴趣可以去享受下探索的乐趣。

Vivado开发工具简介



项目管理：工程设置，新建或添加源文件（约束、代码、仿真源文件），语法模板、IP核生成

Block Design：图形化设计

仿真

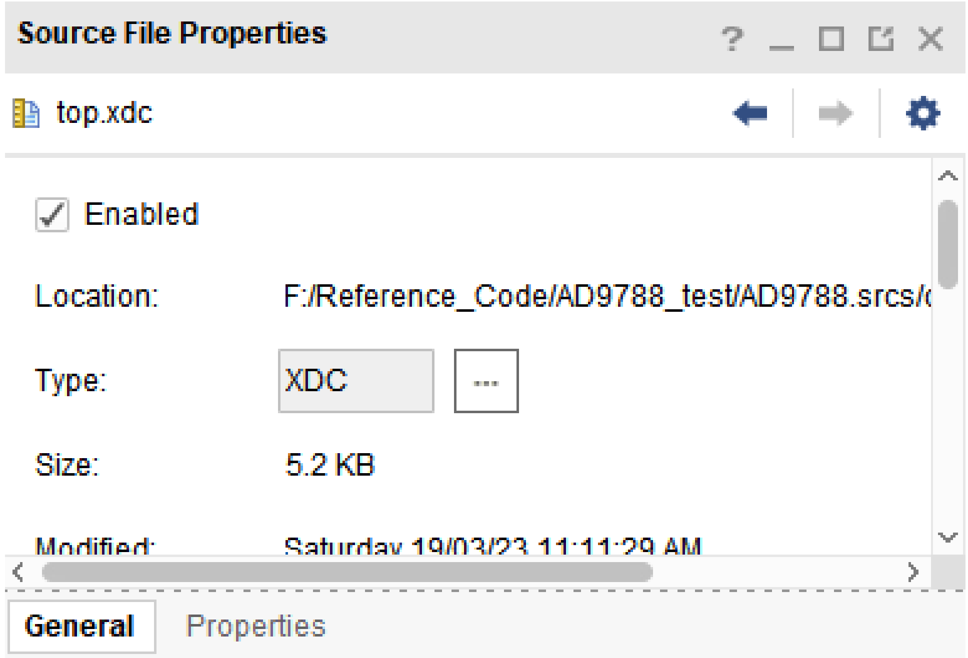
RTL分析：DRC报告，电路原理图

综合：代码综合，时序约束，Set Up Debug，生成门级网表，各种报告

实现：优化，根据网表做映射，布局布线

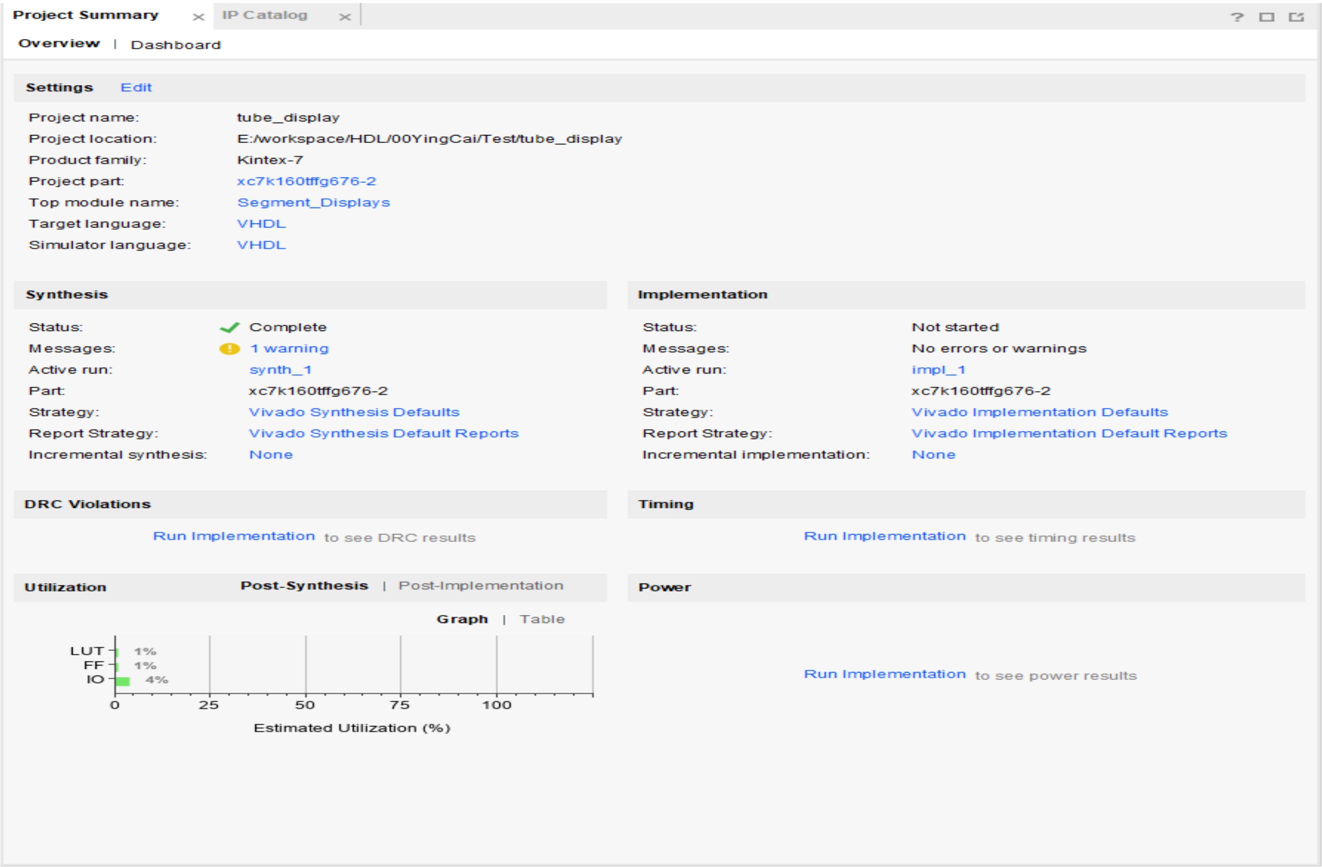
生成比特流以及下载

Vivado开发工具简介



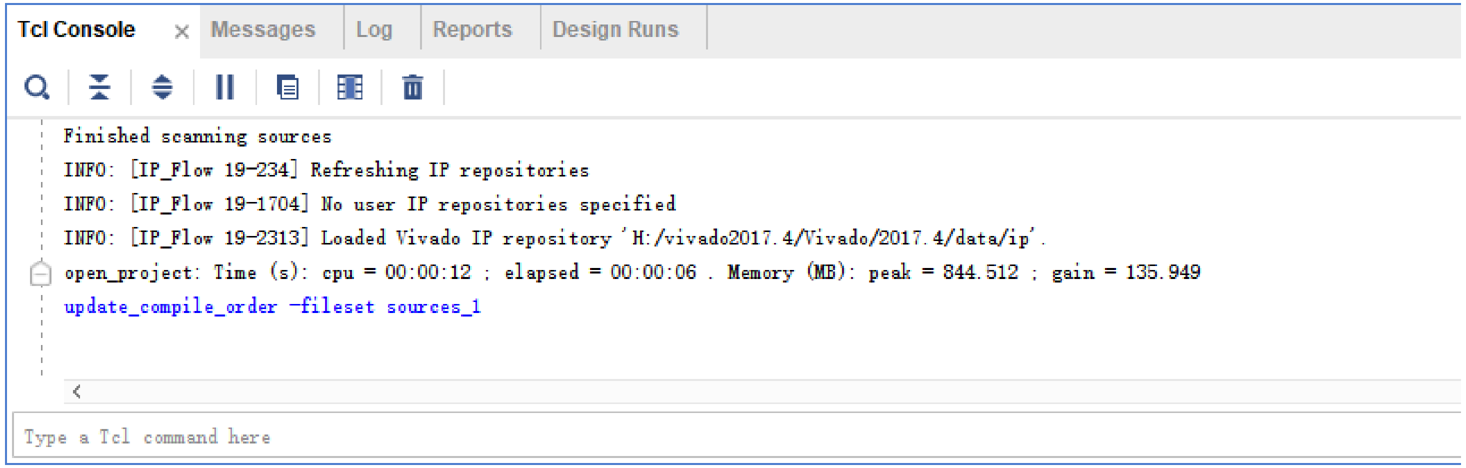
主要显示选中文件的属性。例如，图中显示的是top.xdc约束文件的位置、类型、大小、修改日期等等。可以关掉。

Vivado开发工具简介



工程设置、综合结果、实现结果、DRC、时序、功耗、资源消耗等等。

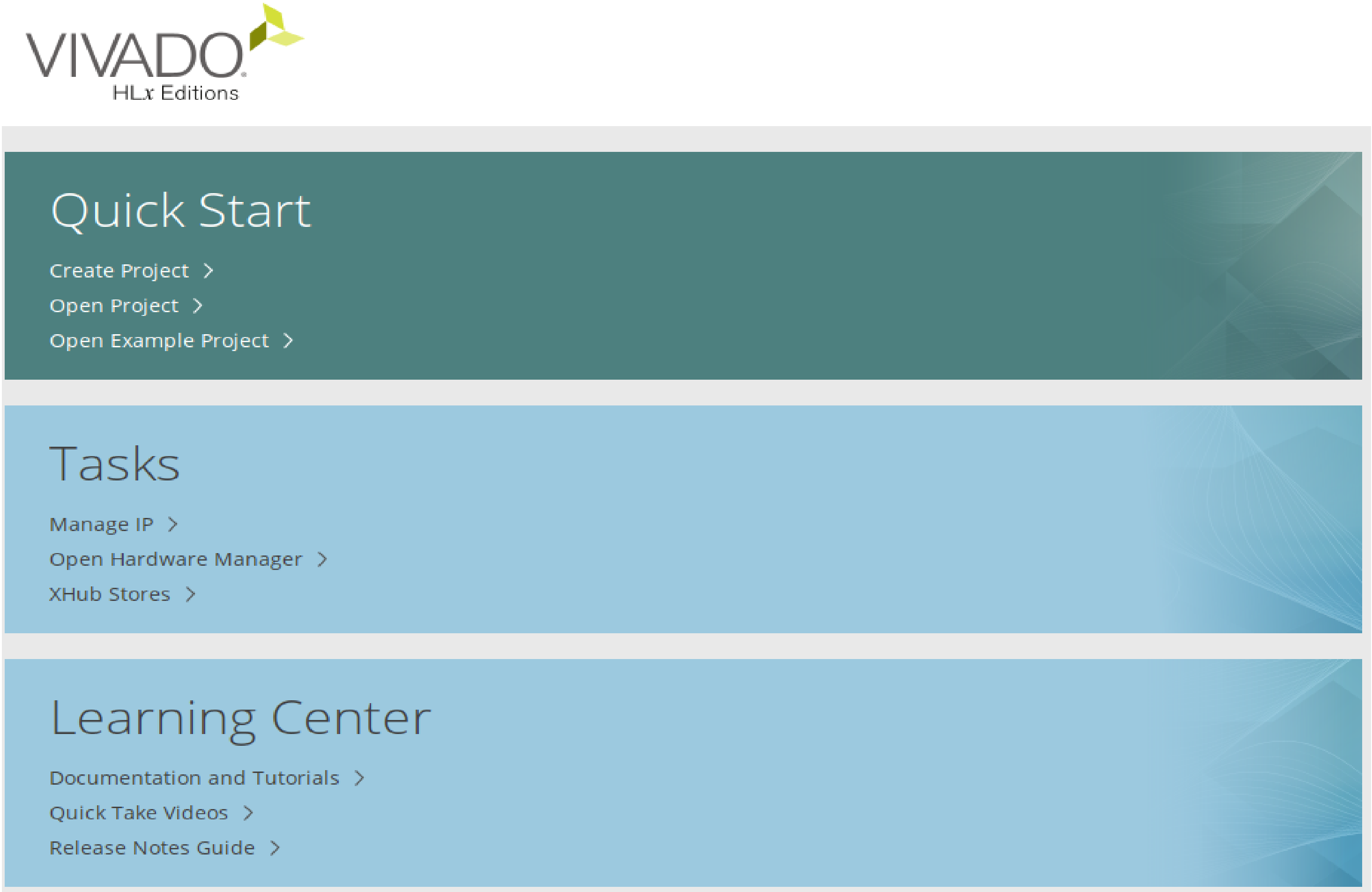
Vivado开发工具简介



Tcl命令窗口、综合信息如警告错误等、报告、一些运行信息

Vivado开发工具简介

第一步： 打开 Vivado，稍等软件启动后，进入如下界面

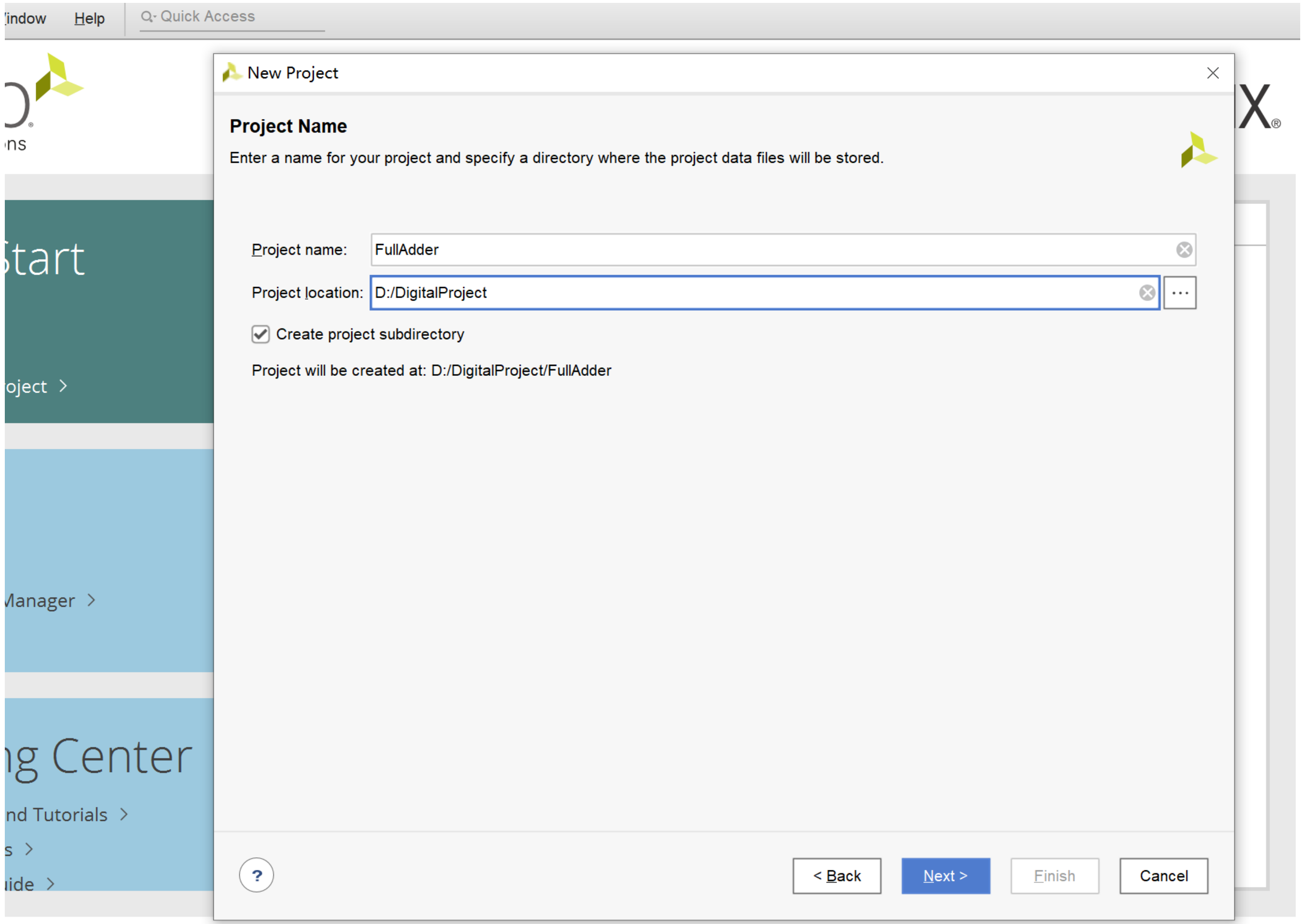


Vivado开发工具简介

第二步：点Create New Project …进入新建工程向导

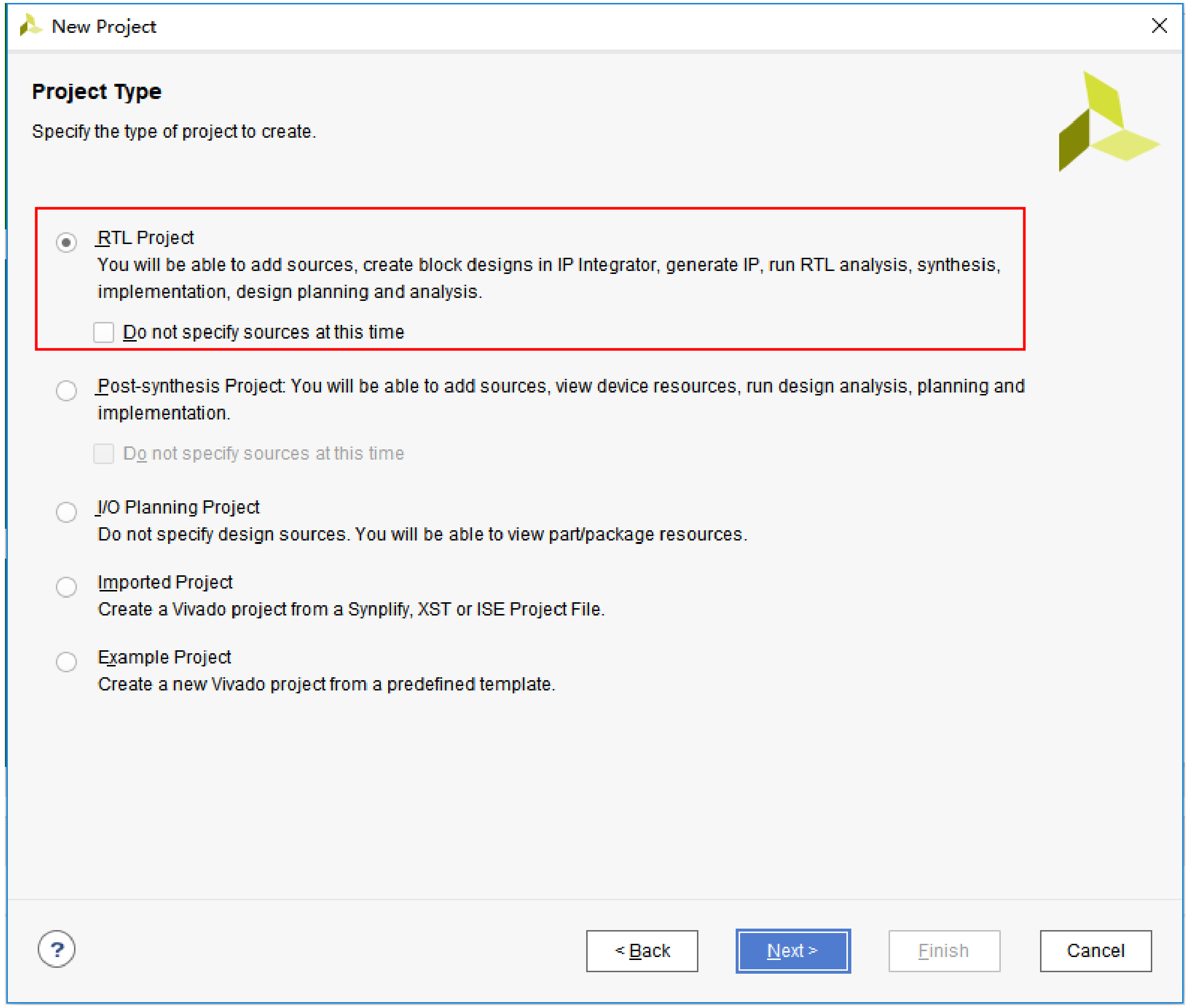
Vivado开发工具简介

第三步： 在弹出窗口填写Project名称，并选择存储地址



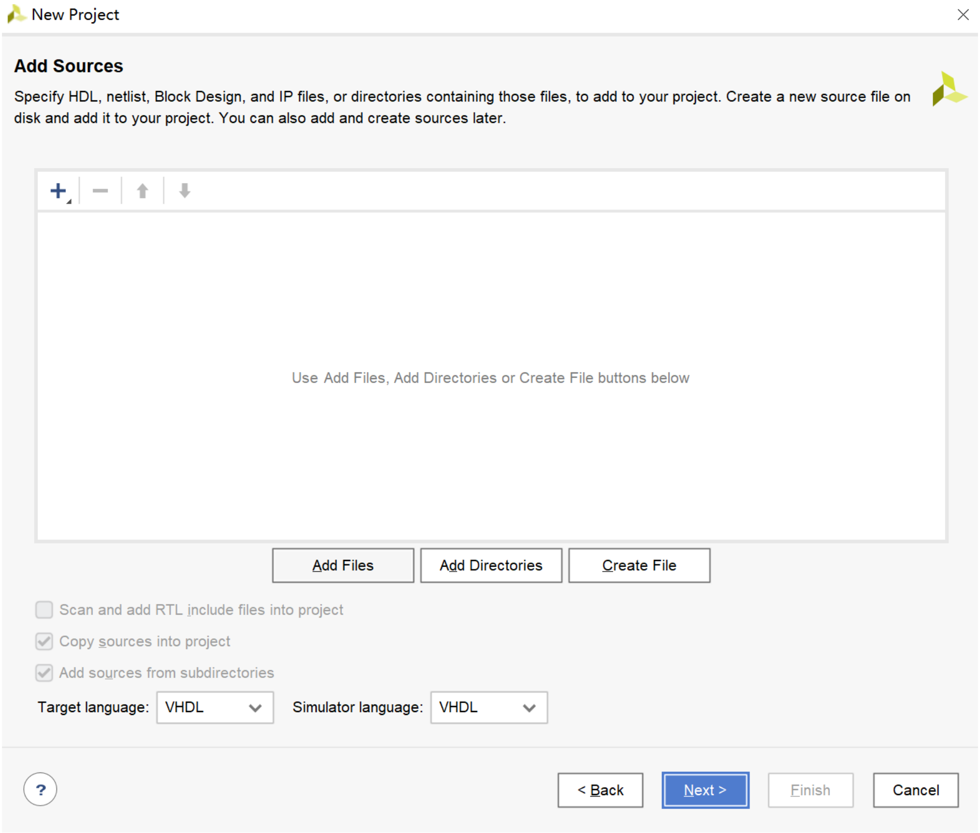
Vivado开发工具简介

在弹出窗口按下图设置：



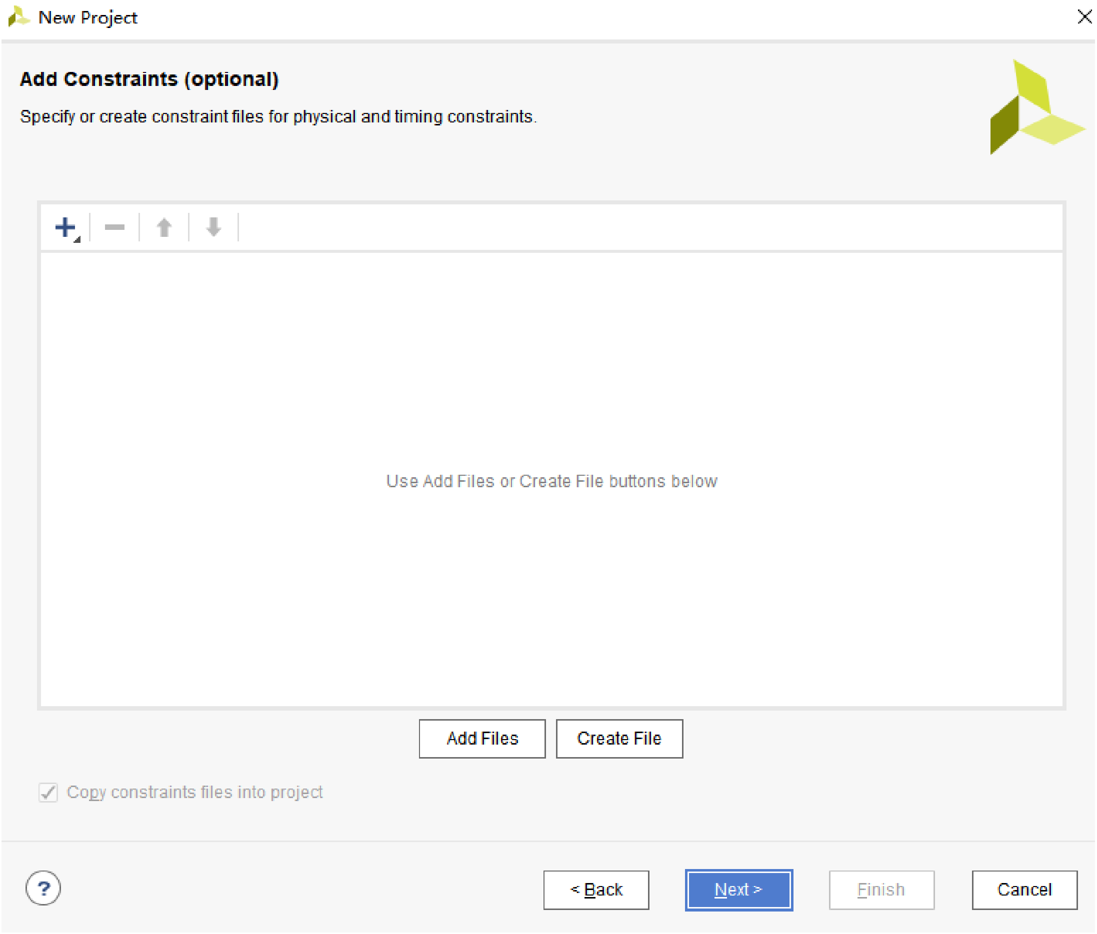
Vivado开发工具简介

由于我们是新建工程，所以此处默认没有可以添加的源文件，并且设置编程语言和仿真语言均设置为VHDL。点击Next



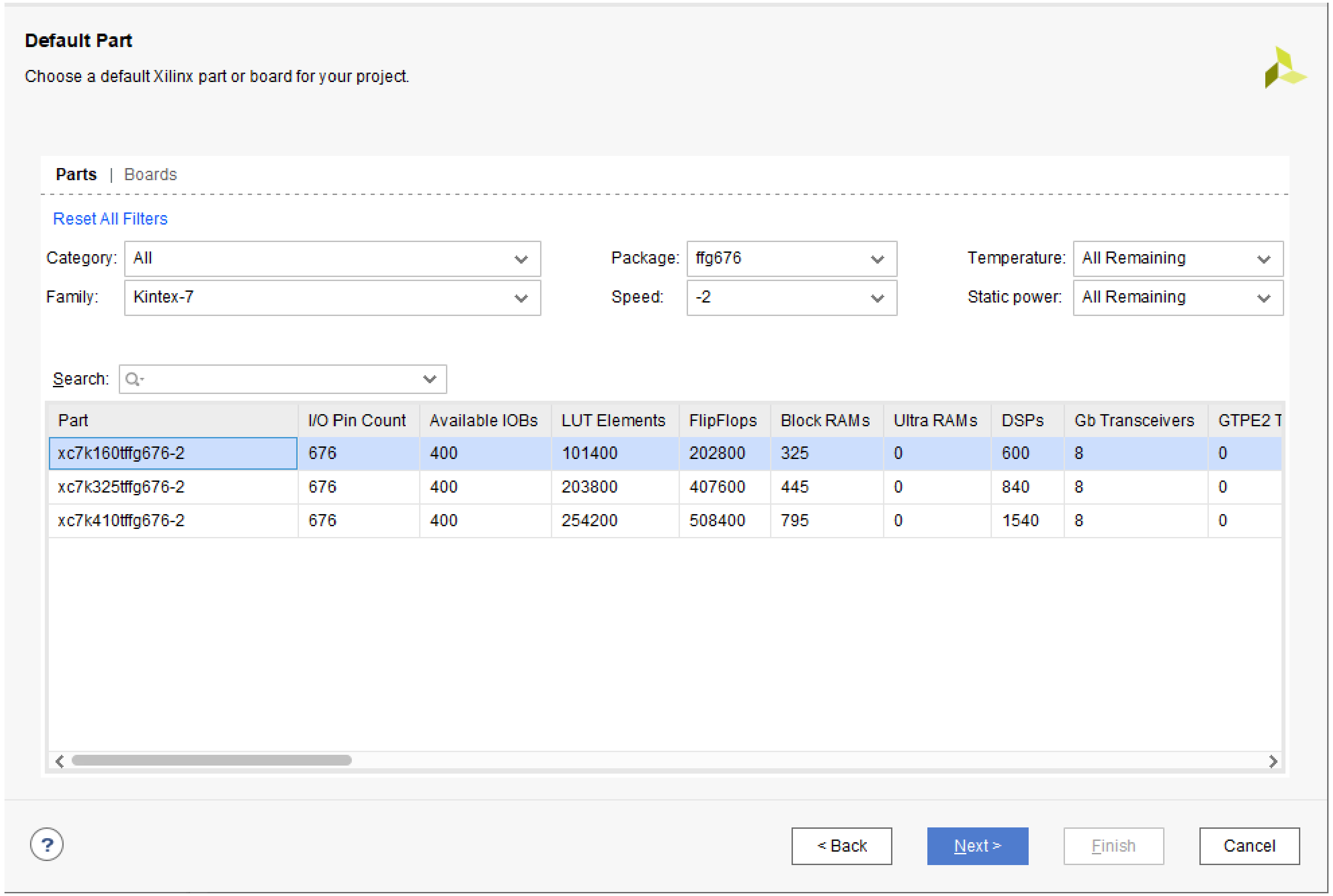
Vivado开发工具简介

也没有可以添加的约束文件，所以不添加，点击Next如下图：



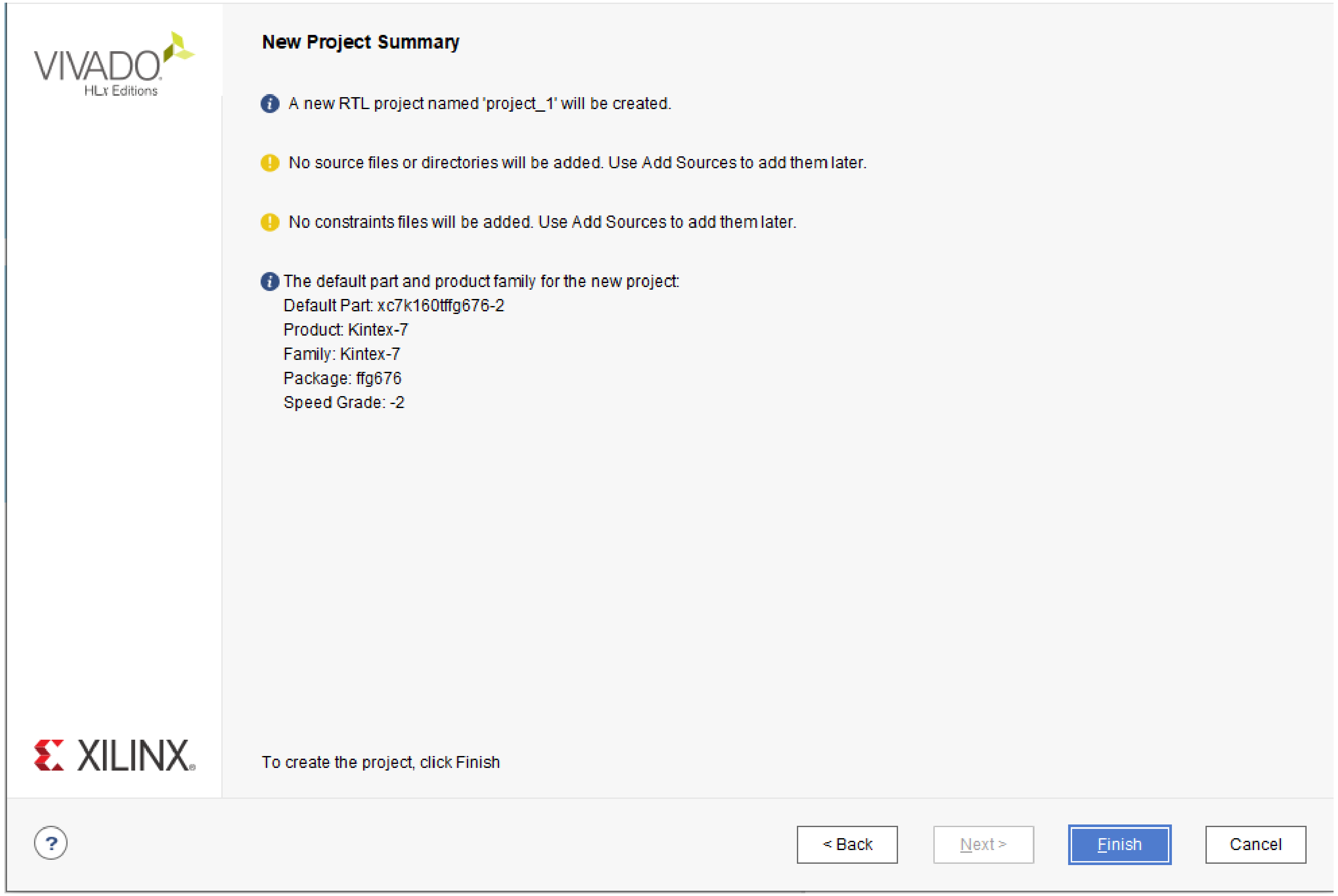
Vivado开发工具简介

选择FPGA芯片，如下图选择xc7k160tffg676-2，并点击下一步



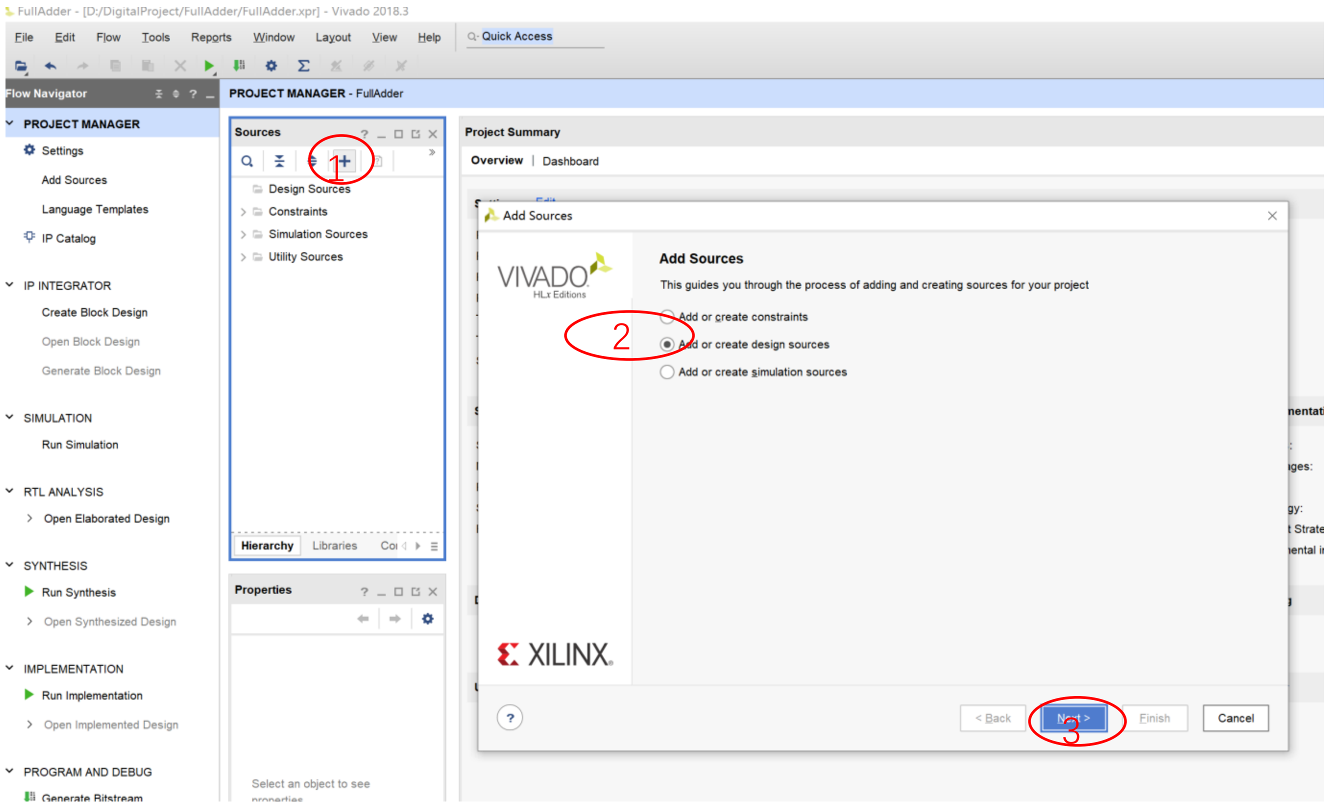
Vivado开发工具简介

查看工程总况，点击Finish



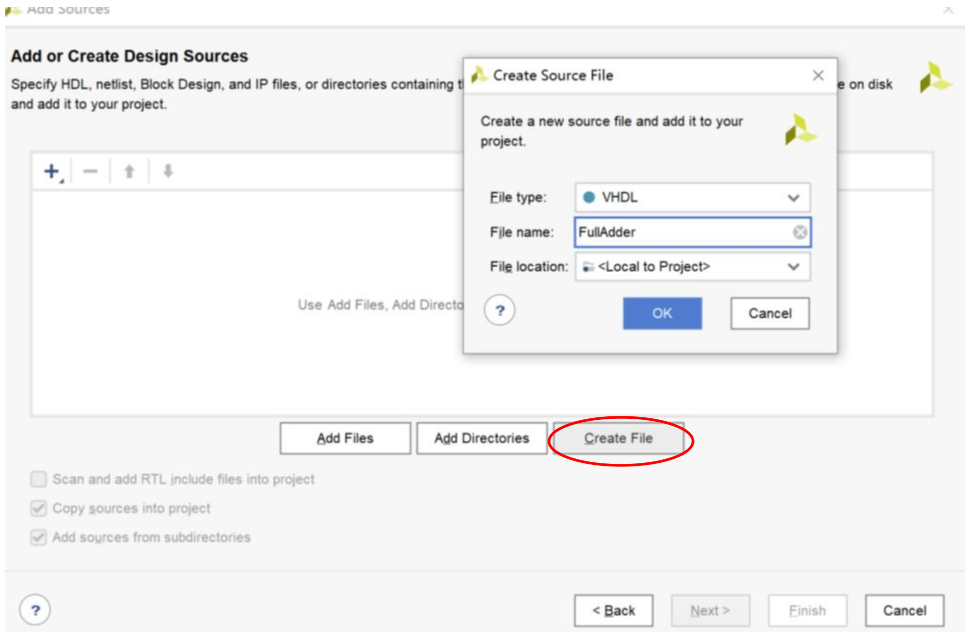
Vivado开发工具简介

第四步： 新建（添加）源文件



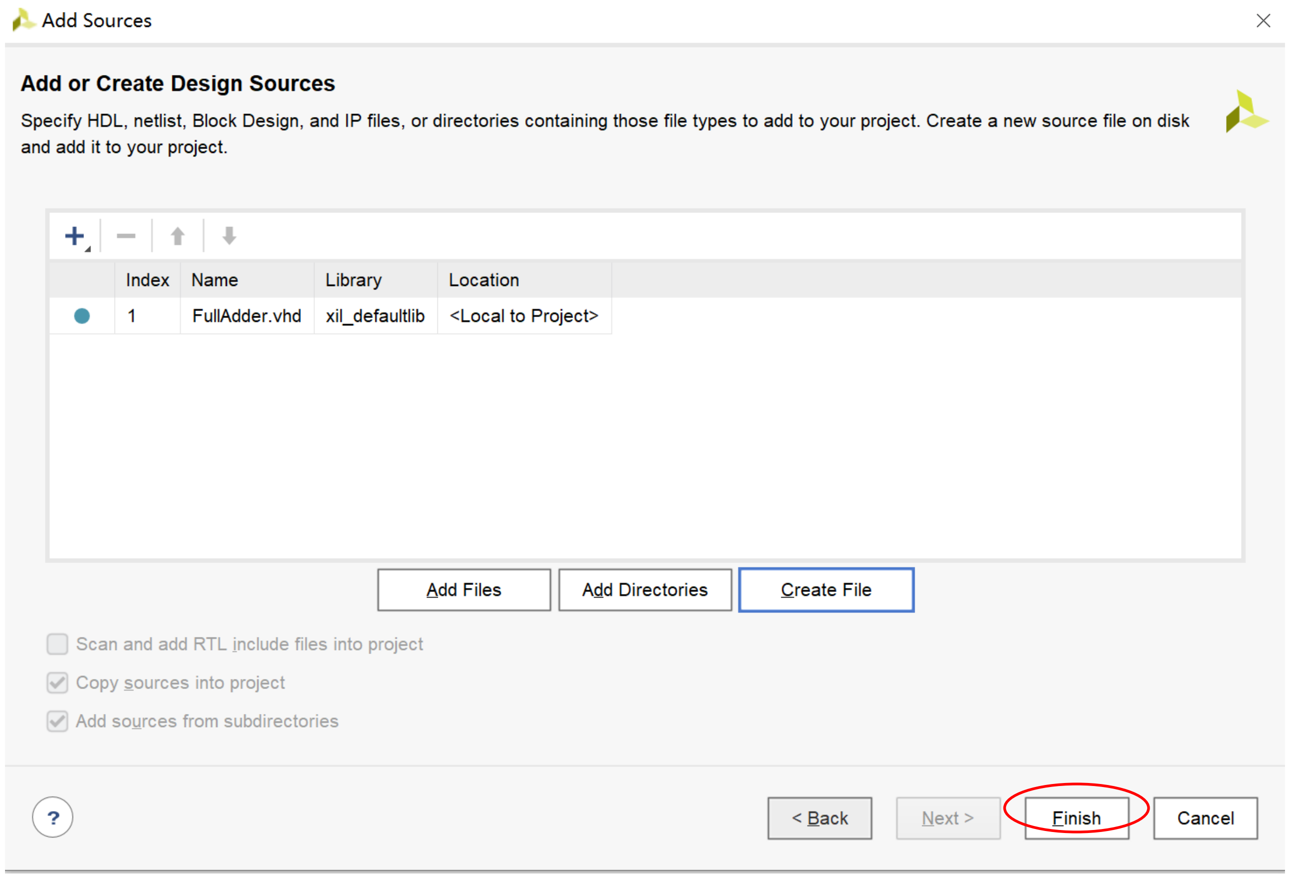
Vivado开发工具简介

在上图中选择Create File…在弹出下面窗口填写新建源文件名称。



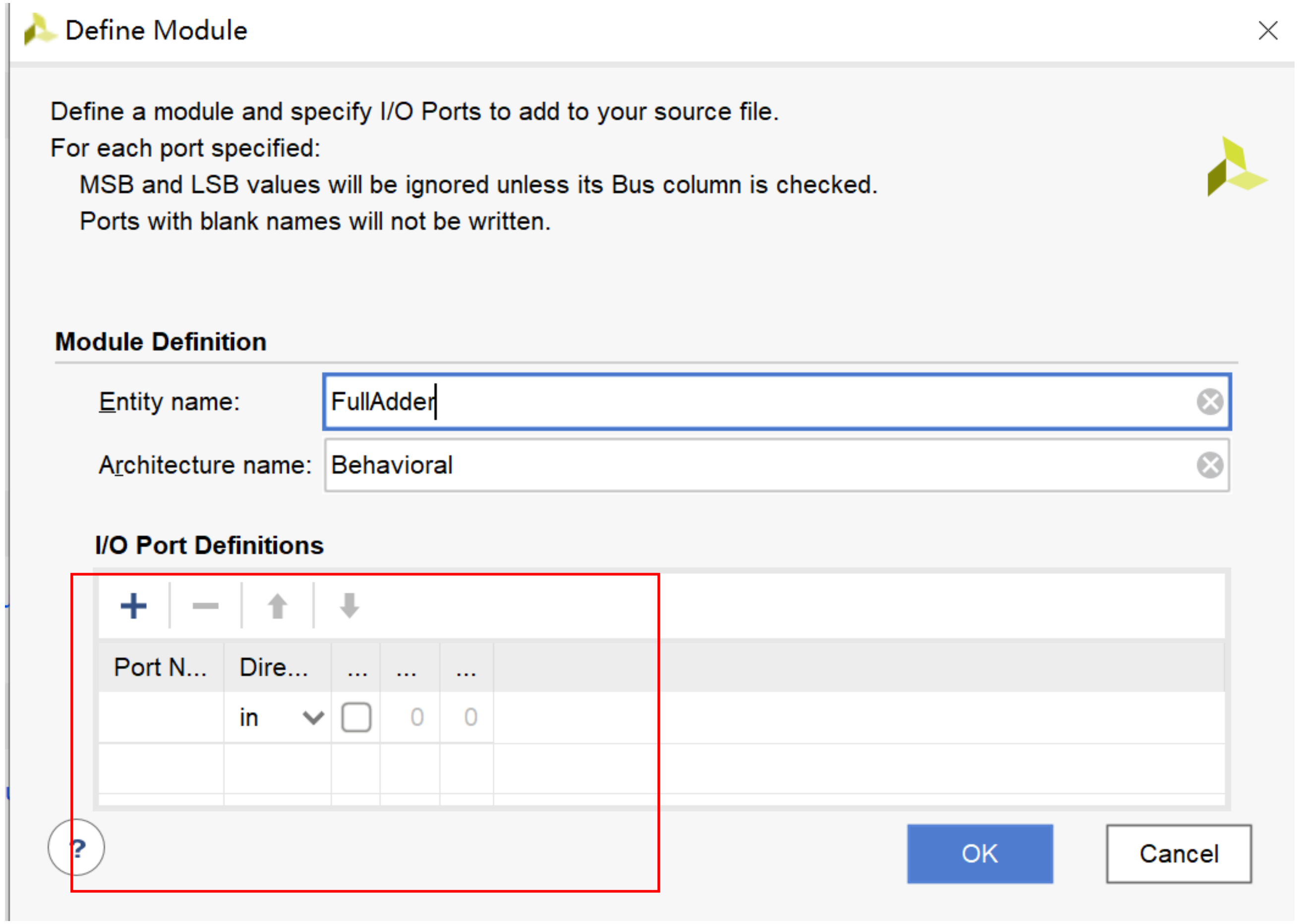
点击OK，弹出下图：

Vivado开发工具简介



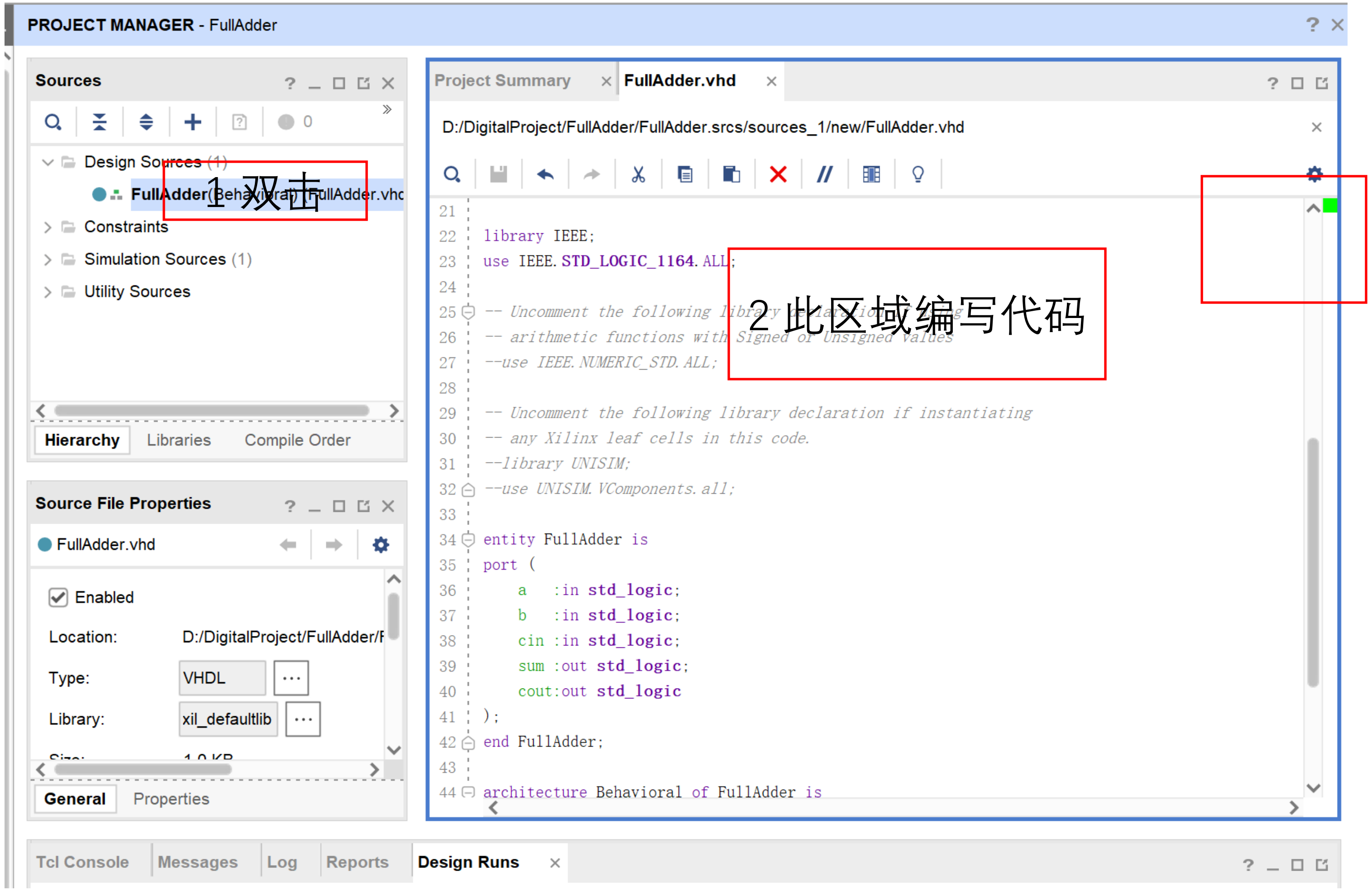
在上图中点击Finish在弹出窗口填写模块名称和端口，如下图所示

Vivado开发工具简介



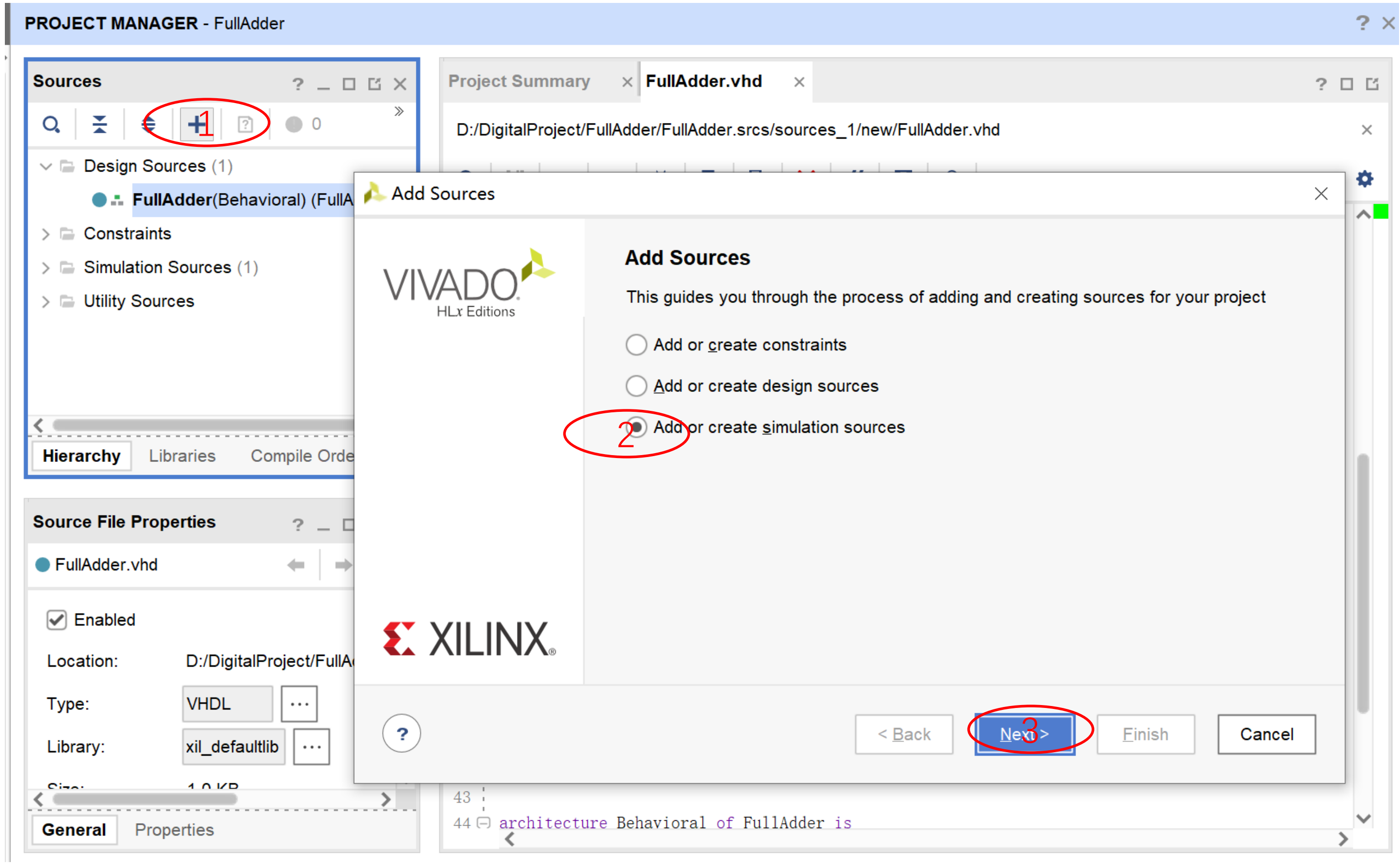
端口名称一般不写，在后面写代码的时候再定义，点击ok，弹出窗口继续点YES

Vivado开发工具简介



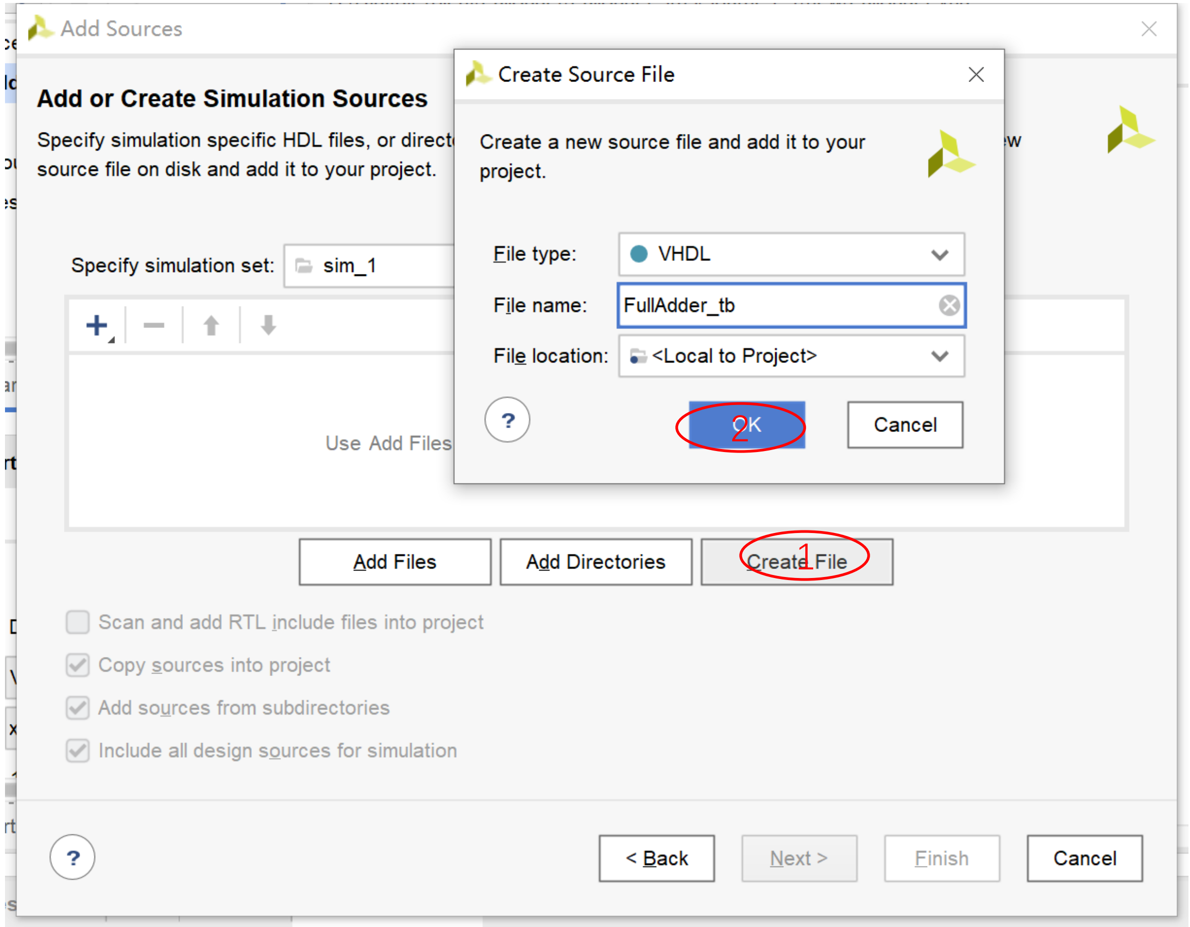
Vivado开发工具简介

第五步：点击加号，选择添加仿真文件



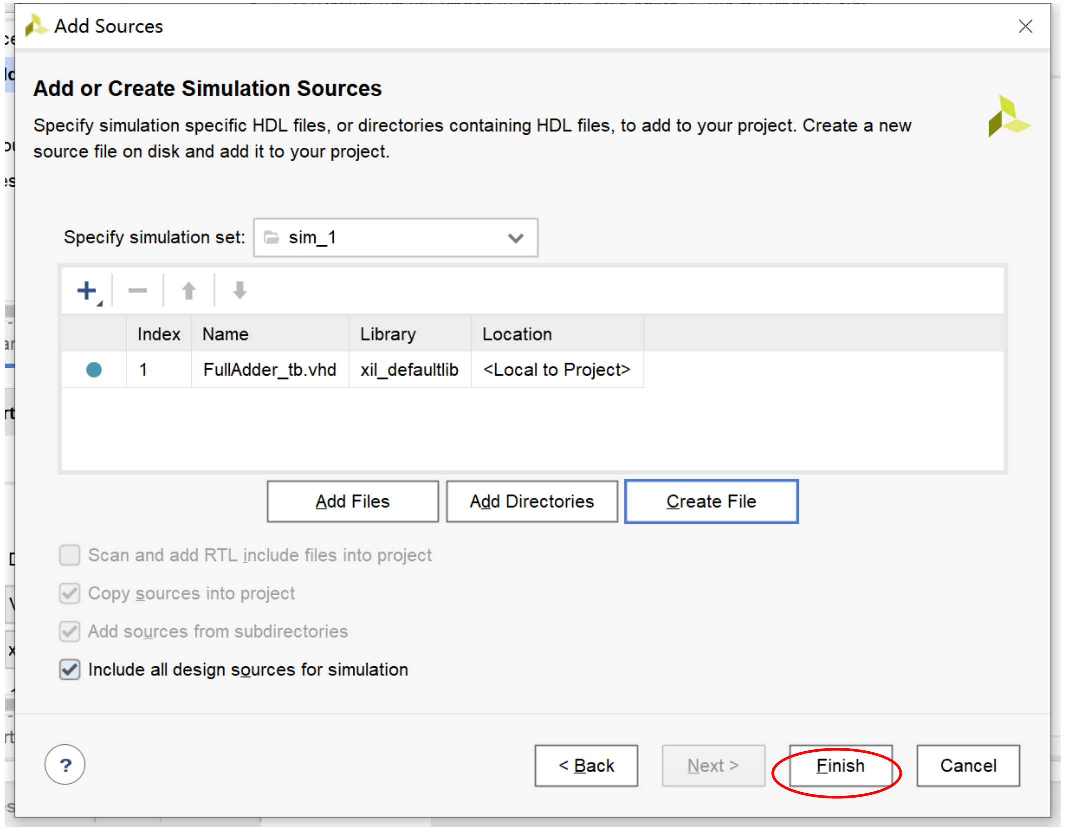
Vivado开发工具简介

在下图的弹出窗口选择Create File…



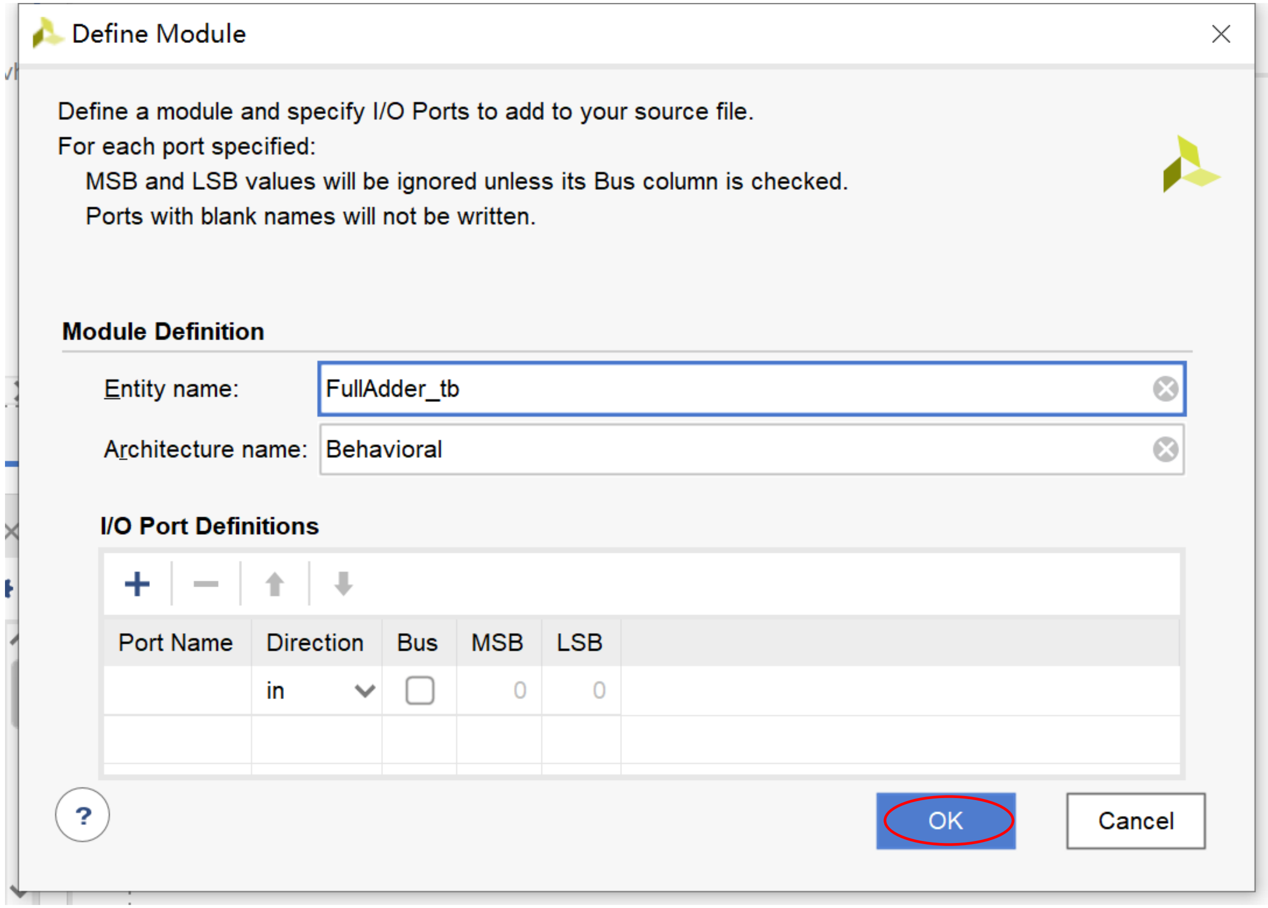
Vivado开发工具简介

点击Finish



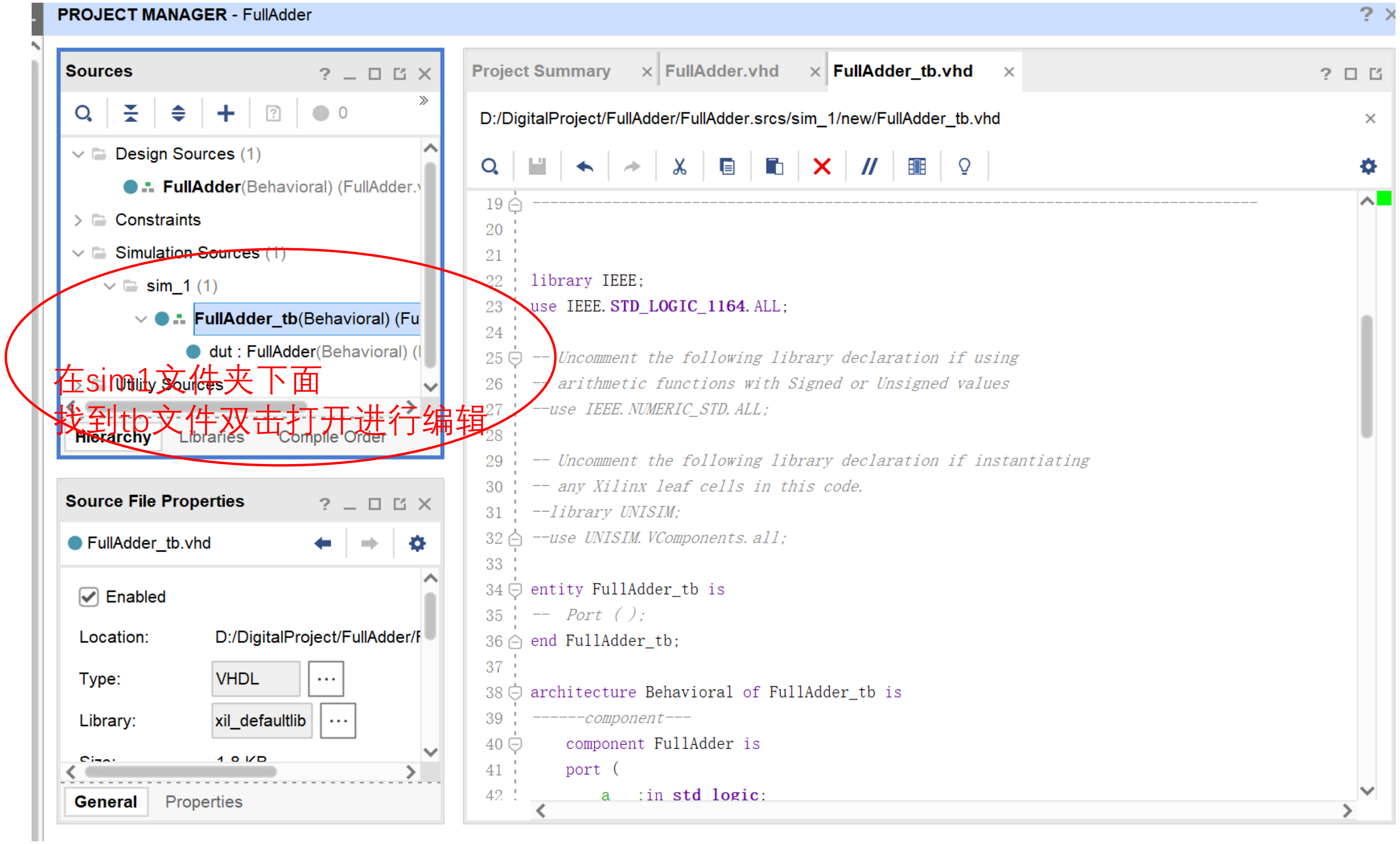
Vivado开发工具简介

点击OK，弹出窗口后然后点YES



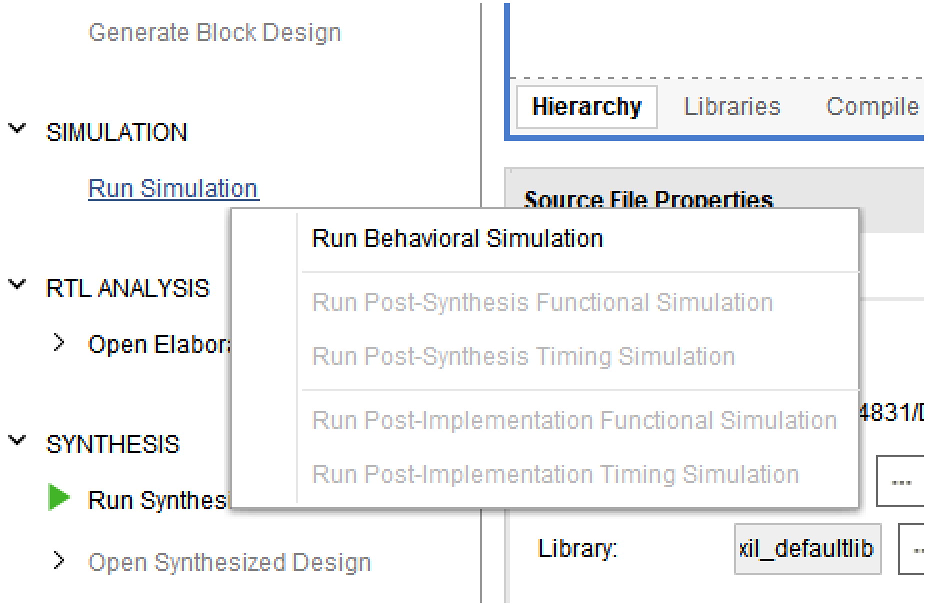
Vivado开发工具简介

双击打开并编写testbench文件



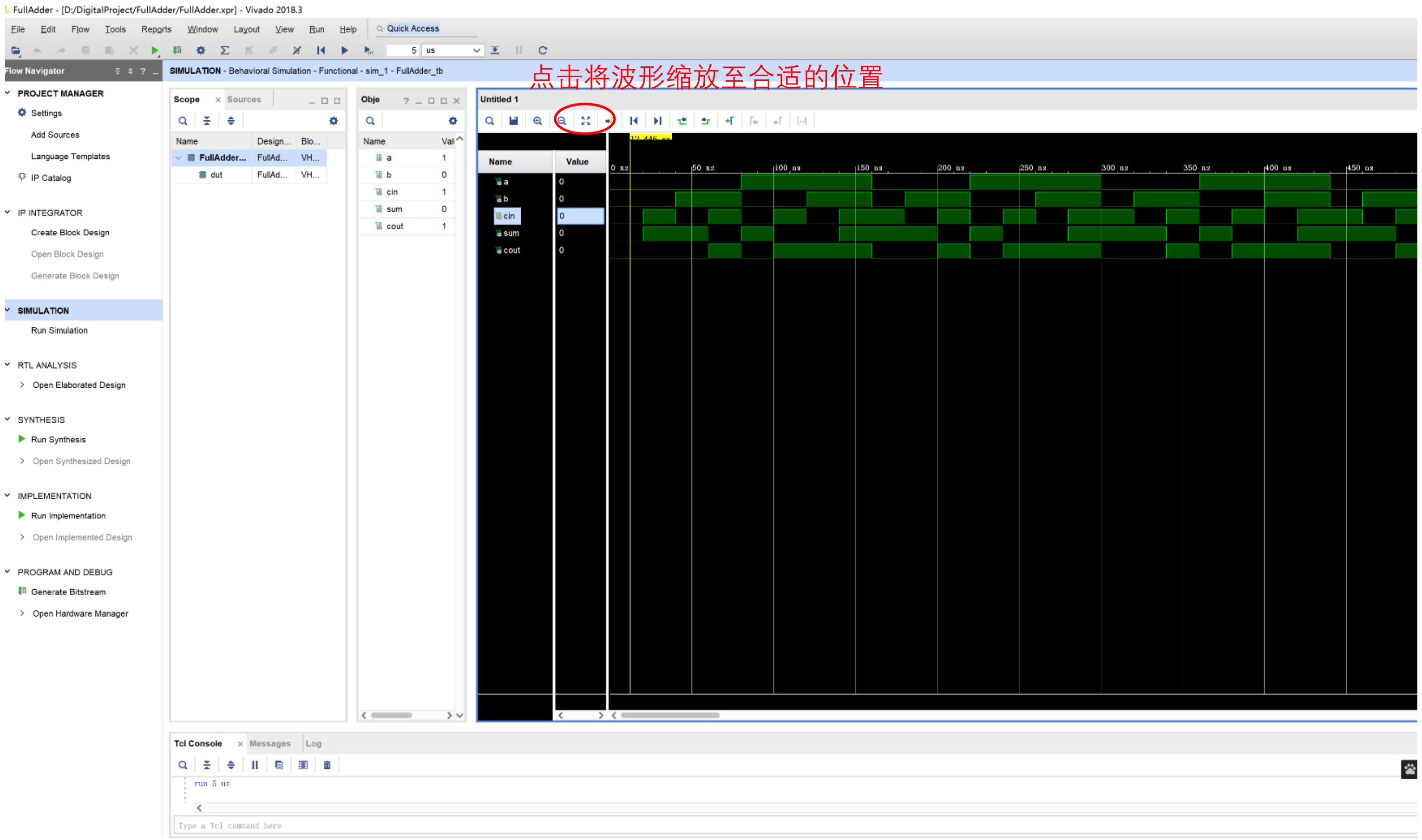
Vivado开发工具简介

编写完仿真源文件后，单击Run Simulation，选择Run Behavior Simulation运行仿真



Vivado开发工具简介

弹出如下的窗口



全加器仿真例程

FullAdder.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder is
port (
        a       :   in  std_logic;
        b       :   in  std_logic;
        cin     :   in  std_logic;
        sum     :   out std_logic;
        cout    :   out std_logic
);
end entity;

architecture Behavioral of FullAdder is

    begin
        sum     <=   a xor b xor cin;
        cout    <=   (a and b) or ((a xor b) and cin);
end architecture;
```

FullAdder_tb.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder_tb is
end entity;

architecture Behavioral of FullAdder_tb is
component FullAdder is
port (
        a       :   in  std_logic;
        b       :   in  std_logic;
        cin     :   in  std_logic;
        sum     :   out std_logic;
        cout    :   out std_logic
);
end component;

signal a    : std_logic;
signal b    : std_logic;
signal cin  : std_logic;
signal sum  : std_logic;
signal cout : std_logic;

    begin
    dut:FullAdder port map(
                        a    =>  a   ,
                        b    =>  b   ,
                        cin  =>  cin ,
                        sum  =>  sum ,
                        cout =>  cout
                      );

    process begin
            a<='0'; b<='0'; cin<='0'; wait for 20 ns;
            a<='0'; b<='0'; cin<='1'; wait for 20 ns;
            a<='0'; b<='1'; cin<='0'; wait for 20 ns;
            a<='0'; b<='1'; cin<='1'; wait for 20 ns;
            a<='1'; b<='0'; cin<='0'; wait for 20 ns;
            a<='1'; b<='0'; cin<='1'; wait for 20 ns;
            a<='1'; b<='1'; cin<='0'; wait for 20 ns;
            a<='1'; b<='1'; cin<='1'; wait for 20 ns;
    end process;

end architecture;
```

VHDL学习扩展资料

https://www.cs.sfu.ca/~ggbaker/reference/std_logic/

The `std_logic` Libraries

The IEEE created the IEEE VHDL library and `std_logic` type in standard 1164. This was extended by Synopsys; their extensions are freely redistributable.

Parts of the IEEE library can be included in an entity by inserting lines like these before your entity declaration:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
```

No attempt has been made here to be definitive or exhaustive. If you want real answers, read the source code. Links to the code or instructions on how to get it are provided for each section. The source is fairly readable to someone who knows some VHDL.

Library information

You can find information about the following libraries here:

- std_logic_1164
- std_logic_arith
- std_logic_unsigned
- std_logic_signed

Missing parts

Some more of the libraries will be added here eventually (hopefully). Until then, I will provide some source code:

- std_logic_entities
- std_logic_components
- std_logic_misc
- std_logic_textio
- gs_types
- cyclone_resolved

About these pages

Return to Greg's References.

实验任务

- 完成一个4比特加法器S = A + B
- 输入：A,B均为4比特二进制补码表征
- 输出：S的位宽自己定(计算溢出？饱和截位？)
- 加法器结构自己定：行波进位，超前进位，进位选择等任意加法器结构
- 自行设计testbench完成RTL行为级仿真
- 验收：以小组为单位完成